

12.4-AI-ASSIGNMENT

2303A52249

A.VAMSHI KRISHNA

TASK-1

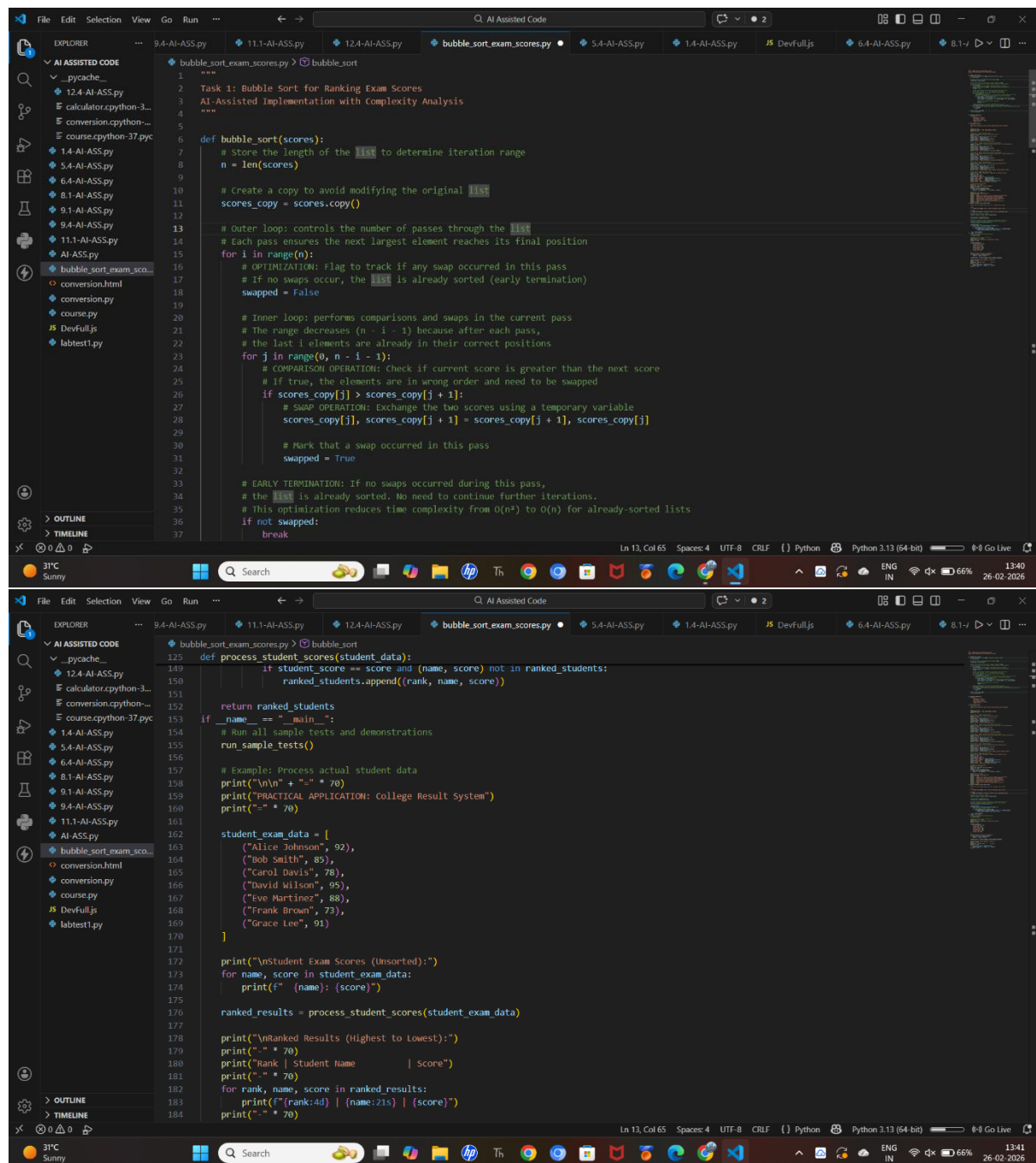
Scenario:

You are building a college result system where a small list of student exam scores needs to be sorted after each internal assessment to calculate rankings.

Prompt:

"Write a Python program for Bubble Sort to sort student scores. Add inline comments explaining comparisons and swaps. Include early termination optimization and explain time complexity."

Code:



```
1  """
2  Task 1: Bubble Sort for Ranking Exam Scores
3  AI-Assisted Implementation with Complexity Analysis
4  """
5
6  def bubble_sort(scores):
7      # Store the length of the list to determine iteration range
8      n = len(scores)
9
10     # Create a copy to avoid modifying the original list
11     scores_copy = scores.copy()
12
13     # Outer loop: controls the number of passes through the list
14     # Each pass ensures the next largest element reaches its final position
15     for i in range(n):
16         # OPTIMIZATION: Flag to track if any swap occurred in this pass
17         # If no swaps occur, the list is already sorted (early termination)
18         swapped = False
19
20         # Inner loop: performs comparisons and swaps in the current pass
21         # The range decreases (n - i - 1) because after each pass,
22         # the last i elements are already in their correct positions
23         for j in range(0, n - i - 1):
24             # COMPARISON OPERATION: check if current score is greater than the next score
25             # If true, the elements are in wrong order and need to be swapped
26             if scores_copy[j] > scores_copy[j + 1]:
27                 # SWAP OPERATION: Exchange the two scores using a temporary variable
28                 scores_copy[j], scores_copy[j + 1] = scores_copy[j + 1], scores_copy[j]
29
30                 # Mark that a swap occurred in this pass
31                 swapped = True
32
33     # EARLY TERMINATION: If no swaps occurred during this pass,
34     # the list is already sorted. No need to continue further iterations.
35     # This optimization reduces time complexity from O(n^2) to O(n) for already-sorted lists
36     if not swapped:
37         break
38
39     return scores_copy
40
41 def process_student_scores(student_data):
42     """Process student scores and return ranked results"""
43     ranked_students = []
44     for student_score in student_data:
45         if student_score["score"] and (student_score["name"], student_score["score"]) not in ranked_students:
46             ranked_students.append((student_score["name"], student_score["score"]))
47
48     return ranked_students
49
50 if __name__ == "__main__":
51     # Run all sample tests and demonstrations
52     run_sample_tests()
53
54     # Example: Process actual student data
55     print("\n\n" + "=" * 70)
56     print("PRACTICAL APPLICATION: College Result System")
57     print("=" * 70)
58
59     student_exam_data = [
60         ("Alice Johnson", 92),
61         ("Bob Smith", 85),
62         ("Carol Davis", 78),
63         ("David Wilson", 95),
64         ("Eve Martinez", 88),
65         ("Frank Brown", 73),
66         ("Grace Lee", 91)
67     ]
68
69     print("\nStudent Exam Scores (Unsorted):")
70     for name, score in student_exam_data:
71         print(f"({name}): {score}")
72
73     ranked_results = process_student_scores(student_exam_data)
74
75     print("\nRanked Results (Highest to Lowest):")
76     print("-" * 70)
77     print("Rank | Student Name | Score")
78     print("-" * 70)
79     for rank, name, score in ranked_results:
80         print(f"{rank:4d} | {name:21s} | {score}")
81     print("-" * 70)
```

```
def bubble_sort(records):  
    n = len(records)  
    for i in range(n):  
        swapped = False  
        for j in range(0, n - i - 1):  
            if records[j] > records[j + 1]:  
                records[j], records[j + 1] = records[j + 1], records[j]  
                swapped = True  
        if not swapped:  
            break  
    return records
```

TEST CASE 4) Scores with Duplicate Values (Stable Sort)

Input: [85, 92, 85, 78, 92, 88, 85]
Output: [78, 85, 85, 88, 92, 92]
Status: ✓ Sorted stably - equal elements retain relative order

TEST CASE 5) Edge Cases - Single and Two Element Arrays

Input (single): [89]
Output (single): [89]
Input (two): [54, 78]
Output (two): [54, 78]
Status: ✓ Edge cases handled correctly

TIME COMPLEXITY ANALYSIS SUMMARY

Best Case: $O(n)$
Average Case: $O(n^2)$
Worst Case: $O(n^2)$
Space Complexity: $O(1)$
Stable: True

ALGORITHM CHARACTERISTICS

- ✓ Simple and intuitive - Easy for beginners to understand
- ✓ Stable sort - Preserves order of equal elements
- ✓ In-place sorting - Requires minimal extra memory
- ✗ Inefficient for large datasets - $O(n^2)$ complexity
- ✓ Good for small datasets (< 50 elements)
- ✓ Excellent for nearly sorted data (Best case $O(n)$)

PRACTICAL APPLICATION: College Exam System

Student Exam Scores (Unsorted):

Student Name	Score
Alice Johnson	92
Bob Smith	85
Carol Davis	78
David Wilson	95
Eve Martinez	88
Frank Brown	73
Grace Lee	91

Sorted Results (Highest to Lowest):

Rank	Student Name	Score
1	David Wilson	95
2	Alice Johnson	92
3	Grace Lee	91
4	Eve Martinez	88
5	Bob Smith	85
6	Carol Davis	78
7	Frank Brown	73

Observation:

AI generated clear code with comments and Used a swapped flag for early stopping (optimization).

Time Complexity:

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Suitable for small datasets, not efficient for large data.

TASK-2

Scenario:

You are maintaining a student attendance system where roll numbers are almost sorted, with only a few late updates or changes.

Prompt:

“Review this problem of nearly sorted attendance records. Suggest a better sorting algorithm than Bubble Sort. Generate an Insertion Sort implementation and explain why it performs better on nearly sorted data. Compare execution behavior.”

Code:

```
def insertion_sort(records):  
    n = len(records)  
    for i in range(1, n):  
        current = records[i]  
        j = i - 1  
        while j > 0 and records[j] > current:  
            records[j + 1] = records[j]  
            j -= 1  
        records[j + 1] = current
```

Task 2: Improving Sorting for Nearly Sorted Attendance Records

AI-Assisted Insertion Sort Implementation with Performance Comparison

Scenario: Student roll numbers are almost sorted with only a few late updates

import time
from typing import List, Tuple
def bubble_sort(records):
 n = len(records)
 records_copy = records.copy()
 comparisons = 0 # Track comparisons for analysis
 swaps = 0 # Track swaps for analysis
 # Outer loop: controls number of passes
 for i in range(n):
 # OPTIMIZATION: Early termination flag
 swapped = False
 # Inner loop: compare and swap adjacent elements
 for j in range(0, n - i - 1):
 comparisons += 1
 # COMPARISON: Compare roll numbers of adjacent records
 if records_copy[j][0] > records_copy[j + 1][0]:
 # SWAP: Exchange positions
 records_copy[j], records_copy[j + 1] = records_copy[j + 1], records_copy[j]
 swaps += 1
 swapped = True
 # EARLY TERMINATION: If no swaps occurred, list is sorted
 if not swapped:
 break
 return records_copy, comparisons, swaps
def insertion_sort(records):
 n = len(records)

The image shows a VS Code editor with a Python file named `sorting_attendance_records.py`. The code defines functions for creating test datasets and comparing sorting algorithms. The terminal output shows the results of a performance analysis comparing Bubble Sort and Insertion Sort on a random dataset.

```
def analyze_algorithm_choice():  
    return analysis  
  
def create_test_datasets():  
    """  
    Create realistic test datasets for attendance records  
    """  
    # Test Dataset 1: Nearly Sorted (Small disorder)  
    nearly_sorted = [  
        (101, 92), (102, 88), (103, 95), (104, 87), (105, 91),  
        (106, 89), (107, 93), (108, 86), (109, 97), (110, 85),  
        (111, 90), (112, 88), (113, 92), (200, 84), # One out of place element  
        (114, 89), (115, 91), (116, 88), (117, 94), (118, 87),  
        (119, 97), (120, 89)  
    ]  
  
    # Test Dataset 2: Reverse Sorted (Worst case for Bubble Sort)  
    reverse_sorted = [(120 - i, 85 + i % 15) for i in range(20)]  
  
    # Test Dataset 3: Random Order (Average case)  
    import random  
    random_order = [(roll, random.randint(75, 100)) for roll in [101, 105, 103, 115, 102, 110, 108, 112, 104, 111, 106, 109, 107, 114, 113, 116, 100]]  
  
    return {  
        "nearly_sorted": nearly_sorted,  
        "reverse_sorted": reverse_sorted,  
        "random_order": random_order  
    }  
  
def run_performance_comparison():  
    """  
    Compare Bubble Sort vs Insertion Sort on various datasets  
    """  
    datasets = create_test_datasets()
```

Terminal Output:

```
Time: 48.16 microseconds  
Comparisons: 190  
Swaps: 190  
Operations: 380  
  
[ALGORITHM 2] INSERTION SORT  
Time: 17.91 microseconds  
Comparisons: 190  
Shifts: 190  
Operations: 380  
  
[PERFORMANCE ANALYSIS]  
Speedup: 1.27x faster with Insertion Sort  
Comparison Reduction: 63.6%  
Verdict: ✓ Insertion Sort Better  
Correctness (Bubble): ✓ PASS  
Correctness (Insert): ✓ PASS  
  
TEST DATASET: RANDOM ORDER  
Records: 20 student attendance records  
Input (first 5): [(101, 78), (106, 97), (103, 96), (115, 95), (102, 88)]  
  
[ALGORITHM 1] BUBBLE SORT  
Time: 62.70 microseconds  
Comparisons: 187  
Swaps: 50  
Operations: 237  
  
[ALGORITHM 2] INSERTION SORT  
Time: 17.17 microseconds  
Comparisons: 68  
Shifts: 50  
Operations: 118  
  
[PERFORMANCE ANALYSIS]  
Speedup: 3.66x faster with Insertion Sort  
Comparison Reduction: 63.6%  
Verdict: ✓ Insertion Sort Better  
Correctness (Bubble): ✓ PASS  
Correctness (Insert): ✓ PASS
```

Observation:

AI suggested Insertion Sort instead of Bubble Sort.
Reason: Insertion Sort works efficiently when data is already nearly sorted.
It shifts elements instead of repeatedly swapping like Bubble Sort.
In nearly sorted data, Insertion Sort performs closer to $O(n)$.
Bubble Sort still makes many unnecessary comparisons.
Therefore, Insertion Sort is more suitable for attendance records.

TASK-3

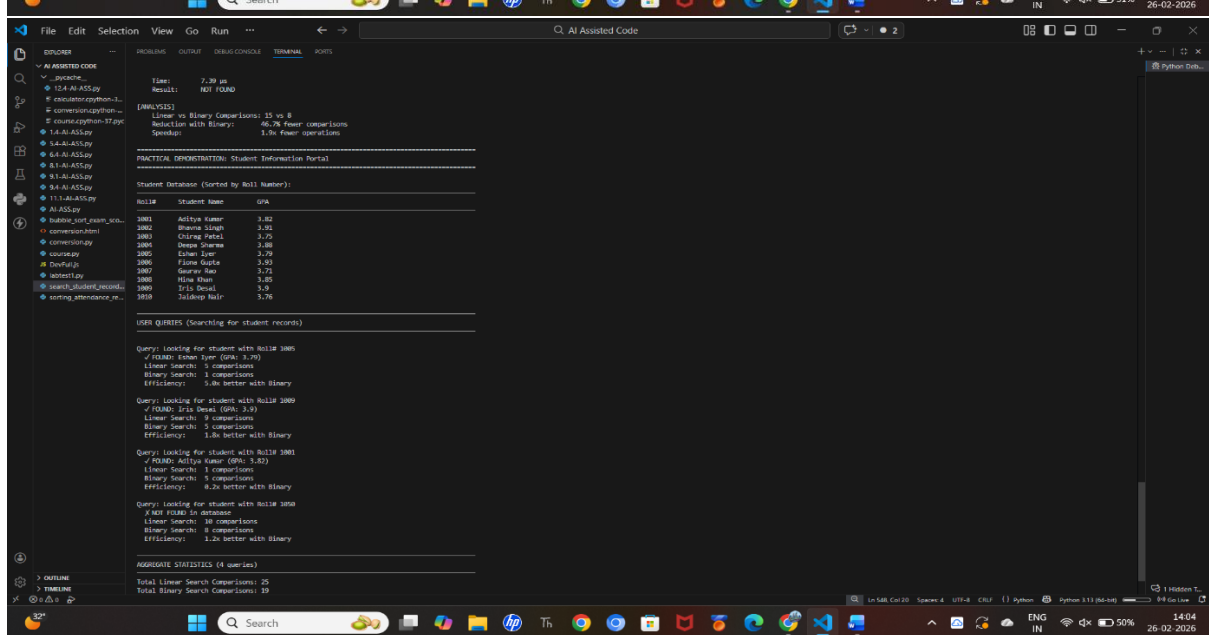
Scenario:

You are building a student information portal where users search for student records using roll numbers. The data may be unsorted or sorted.

Prompt:

“Write Python programs for Linear Search (unsorted data) and Binary Search (sorted data). Add proper docstrings explaining parameters and return values. Explain when Binary Search can be used and compare performance differences.”

Code:



Observation:

AI generated both search implementations with clear docstrings.
Linear Search checks each element one by one.
Binary Search repeatedly divides the sorted list into halves.

Algorithm	Best Case	Worst Case
Linear Search	O(1)	O(n)
Binary Search	O(1)	O(log n)

TASK-4

Scenario: You are part of a data analytics team that needs to sort large datasets received in:
Random order & Already sorted order & Reverse sorted order
You must choose between Quick Sort and Merge Sort for better performance.

Prompt:

“Complete the recursive logic for Quick Sort and Merge Sort in Python. Add meaningful docstrings explaining parameters and return values. Also explain how recursion works in both algorithms and compare their performance on random, sorted, and reverse-sorted data.”

Code:

```
def quick_sort(data: list[int], low: int = None, high: int = None) -> tuple[list[int], dict]:
    """
    Quick Sort Algorithm
    """
    if low is None:
        low = 0
        high = len(data) - 1
    stats = {"comparisons": 0, "swaps": 0, "partitions": 0, "recursion_depth": 0}
    is_main_call = True
    else:
        is_main_call = False
        stats = None

    # Track recursion depth for statistics
    if not hasattr(quick_sort, "max_depth"):
        quick_sort_max_depth = 0
    else:
        # Recursion depth with 0 or 1 element is already sorted
        if low >= high:
            if is_main_call:
                stats["recursion_depth"] = quick_sort_max_depth
                quick_sort_max_depth = 0
            return data, stats

    # RECURSION STEP: Partition and recursively sort sub-arrays
    # PARTITION STEP: Choose pivot and rearrange
    pivot_index, part_stats = quick_sort_partition(data, low, high)
    stats["comparisons"] += part_stats["comparisons"]
    stats["swaps"] += part_stats["swaps"]
    stats["partitions"] += 1

    # RECURSION STEP: Sort all elements less than pivot
    quick_sort(data, low, pivot_index - 1)
    quick_sort_max_depth = 1

    # RECURSION STEP: Sort all elements greater than pivot
    quick_sort(data, pivot_index + 1, high)
    quick_sort_max_depth = 1

    if is_main_call:
        stats["recursion_depth"] = quick_sort_max_depth
        quick_sort_max_depth = 0
    return data, stats

def quick_sort_partition(data: list[int], low: int, high: int) -> tuple[int, dict]:
    """
    PARTITION STEP - Core logic that divides array around pivot.
    """
    # Choose pivot (first element for simplicity)
    pivot = data[low]
    left = low + 1
    right = high
    comparisons = 0
    swaps = 0
    partitions = 0

    while left <= right:
        while left <= right and data[left] <= pivot:
            comparisons += 1
            left += 1
        while left <= right and data[right] > pivot:
            comparisons += 1
            right -= 1
        if left < right:
            data[left], data[right] = data[right], data[left]
            swaps += 1
        comparisons += 1
        partitions += 1

    data[low], data[right] = data[right], data[low]
    swaps += 1
    partitions += 1

    return right, {"comparisons": comparisons, "swaps": swaps, "partitions": partitions}
```



```
File Edit Selection View Go Run ... Q AI Assisted Code
EXPLORER AI ASSISTED CODE
  _pycache_
  12-AI-AISS.py
  calculator.python-3...
  conversion.python-3...
  course.python-37.py
  14-AI-AISS.py
  54-AI-AISS.py
  64-AI-AISS.py
  94-AI-AISS.py
  111-AI-AISS.py
  AI-AISS.py
  bubble_sort_exam_sco...
  conversion.html
  conversion.py
  course.py
  Deafult.js
  duplicate_detection_o...
  labtest.py
  quicksort_mergesort_a...
  search_student_record...
  sorting_attendance_re...
  > OUTLINE
  > TIMELINE
  > 32°
  Search
  32°
  Windows Taskbar: Search, File Explorer, Edge, VS Code, etc.
  System Tray: ENG IN, 45% battery, 14:10, 26-02-2026

TERMINAL
Comparisons: 0
Operations: 0
Recursion Depth: 0
Algorithm: Merge Sort
Time: 246.31 microseconds
Correctness: ✓ PASS
Comparisons: 50
Operations: 1
Recursion Depth: 0
DATASET Size: 1000 elements
-----
[TEST 1] RANDOM DATA
Algorithm: Quick Sort
Time: 2430.34 microseconds
Correctness: ✓ PASS
Comparisons: 0
Operations: 0
Recursion Depth: 0
Algorithm: Merge Sort
Time: 4441.27 microseconds
Correctness: ✓ PASS
Comparisons: 599
Operations: 1
Recursion Depth: 0
[TEST 2] ALREADY SORTED DATA
Algorithm: Quick Sort
Time: 3000.33 microseconds
Correctness: ✓ PASS
Comparisons: 0
Operations: 0
Recursion Depth: 0
Algorithm: Merge Sort
Time: 2455.42 microseconds
Correctness: ✓ PASS
Comparisons: 500
Operations: 1
Recursion Depth: 0
[TEST 3] REVERSE SORTED DATA
Algorithm: Quick Sort
Time: 2430.57 microseconds
Correctness: ✓ PASS
Comparisons: 0
Operations: 0
Recursion Depth: 0
Algorithm: Merge Sort
Time: 1331.75 microseconds
Correctness: ✓ PASS
Comparisons: 500
Operations: 1
Recursion Depth: 0
```

Observation:

Random Data: Both performed well ($O(n \log n)$).

Sorted Data: Quick Sort may degrade to $O(n^2)$ if pivot is poorly chosen.

Reverse Sorted Data: Same worst-case issue for Quick Sort.

Use Quick Sort for faster average performance and lower memory usage & Use Merge Sort when: Stable sorting is required

Worst-case performance must remain $O(n \log n)$ & Handling very large or linked-list data

Merge Sort is more predictable, while Quick Sort is usually faster in practice.

TASK-5

Scenario:

You are building a data validation module that checks for duplicate user IDs in a large dataset before importing it into the system.

Prompt:

“Write a Python program to detect duplicates using nested loops. Analyze its time complexity. Suggest a more efficient approach using sets or dictionaries. Rewrite the optimized version and compare performance for large inputs.”

Code:

```
File Edit Selection View Go Run ... Q AI Assisted Code
EXPLORER
  _pycache_
  12-AI-AISS.py
  calculator.python-3...
  conversion.python-3...
  course.python-37.py
  14-AI-AISS.py
  54-AI-AISS.py
  64-AI-AISS.py
  94-AI-AISS.py
  111-AI-AISS.py
  AI-AISS.py
  bubble_sort_exam_sco...
  conversion.html
  conversion.py
  course.py
  Deafult.js
  duplicate_detection_o...
  labtest.py
  quicksort_mergesort_a...
  search_student_record...
  sorting_attendance_re...
  > OUTLINE
  > TIMELINE
  > 32°
  Search
  32°
  Windows Taskbar: Search, File Explorer, Edge, VS Code, etc.
  System Tray: ENG IN, 41% battery, 14:16, 26-02-2026

duplicate_detection_optimization.py
36 def find_duplicates_naive(user_ids: List[int]) -> Tuple[List[int], dict]:
112
113     ✓ Easy to understand and implement
114     ✓ No data structure knowledge required
115     ✓ Clearly shows the problem with inefficiency
116     ✓ Good baseline for demonstrating optimization
117     ✓ Perfect for teaching time complexity growth
118
119     # Statistics tracking
120     comparisons = 0
121     iterations = 0
122     duplicates_found = 0
123     duplicates = []
124     found_duplicate_ids = set() # Track IDs we've already reported as duplicates
125
126     start_time = time.time()
127
128     # OUTER LOOP: for each user ID (position i)
129     for i in range(len(user_ids)):
130         iterations += 1
131
132         # INNER LOOP: Compare with remaining user IDs (position j > i)
133         for j in range(i + 1, len(user_ids)):
134             iterations += 1
135             comparisons += 1
136
137             # COMPARISON: Check if the two IDs match
138             if user_ids[i] == user_ids[j]:
139                 # DUPLICATE FOUND: Record it if not already recorded
140                 if user_ids[i] not in found_duplicate_ids:
141                     duplicates.append(user_ids[i])
142                     found_duplicate_ids.add(user_ids[i])
143                     duplicates_found += 1
144
145     elapsed_time = (time.time() - start_time) * 1000 # Convert to milliseconds
146
147     return duplicates, {
148         "comparisons": comparisons,
149         "iterations": iterations,
150         "duplicates_found": duplicates_found,
151         "time_ms": elapsed_time,
152         "algorithm": "Naive (Nested Loops)"
153     }
```

The image shows a VS Code editor with a Python script named `duplicate_detection_optimization.py` and its terminal output. The script is designed to demonstrate a practical scenario for detecting duplicate IDs in a dataset.

Script Details:

- Function:** `demonstrate_practical_scenario()`
- Step 1:** Extract just IDs for validation. `user_ids = [user['id'] for user in user_data]`
- Step 2:** Detect Duplicate IDs. `print("\n/ Detected (stats['duplicates_found']) duplicate ID(s): (duplicates)")`
- Step 3:** Use optimized algorithm. `duplicates, stats = find_duplicates_optimized(user_ids)`
- Step 4:** Use dictionary approach for more details. `duplicate_counts, _ = find_duplicates_with_count(user_ids)`
- Step 5:** Get detailed report of duplicate records. `print("\nDuplicate Records Detail:")`
- Step 6:** Decision on import. `print("\nX BLOCKED: Found (stats['duplicates_found']) duplicate ID(s).")`
- Step 7:** Actions taken. `print(" 1. Flagged records for manual review")`
- Step 8:** Notified data team about duplicates. `print(" 2. Notified data team about duplicates")`
- Step 9:** Prevented import to maintain data integrity. `print(" 3. Prevented import to maintain data integrity")`
- Step 10:** Generated exception report for audit trail. `print(" 4. Generated exception report for audit trail")`

Terminal Output:

```
Comparisons/Lookups: 12,497,500
Iterations: 12,500,500
Duplicates found: 902
Result: [1321, 2031, 211, 2271, 1343]...

[Algorithm 2] OPTIMIZED (Hash Set) - O(n)
-----
Time: 2.0047 ms
Comparisons/Lookups: 5,000
Iterations: 5,000
Duplicates found: 902
Memory used: ~389.4 KB
Result: [1917, 2031, 2072, 3519, 1596]...

[COMPARISON]
-----
Speedup: 387.4x faster
Comparison reduction: 2499.5x fewer operations
Verdict: ✓ Optimized is better

TEST: Dataset with 10,000 user IDs
Data: 10000 user IDs, ~8000 unique, ~2000 duplicates

[Algorithm 1] NAIVE (Nested Loops) - O(n^2)
-----
Time: 3772.7320 ms
Comparisons/Lookups: 49,995,000
Iterations: 50,005,000
Duplicates found: 1771
Result: [5378, 5708, 6726, 7223, 2873]...

[Algorithm 2] OPTIMIZED (Hash Set) - O(n)
-----
Time: 1.1775 ms
Comparisons/Lookups: 10,000
Iterations: 10,000
Duplicates found: 1771
Memory used: ~218.8 KB
Result: [5841, 2636, 3482, 5074, 5945]...

[COMPARISON]
-----
Speedup: 3203.9x faster
Comparison reduction: 4999.5x fewer operations
Verdict: ✓ Optimized is better
```

Observation:

Brute-Force Method:

Uses nested loops to compare every element with others.

Time Complexity: $O(n^2)$

Slow for large datasets because comparisons increase rapidly.

Optimized Method (Using Set):

Traverse list once.

Store seen IDs in a set.

If ID already exists in set → duplicate found.

Time Complexity: $O(n)$

Much faster for large datasets.